

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Vishwa.U	Reg. No.:	192511192
Section / Batch:	SLOT-B / 2025	Date:	3/08/26

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floatingpoint representation. Given a decimal number (e.g., -13.25), analyse how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct

I VISHWA U (192511192) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

CSA1220 – Computer Architecture

Assessment Tool 1 – Analytical Problem Solving (CO2)

Question 1: 2's Complement Addition & Subtraction (+45 and -23)

Step 1: Convert to 8-bit binary

- +45 → 00101101
- 23 in binary = 00010111 → -23 in 2's complement = invert bits (11101000) + 1 = 11101001

Step 2: Addition — (+45) + (-23)

```
00101101  (+45)
+ 11101001  (-23)
-----
100010110  (9-bit raw sum)
```

Discard the carry-out of the MSB (standard rule in 2's complement addition): result = 00010110 = 22 (decimal).

Check: $45 - 23 = 22$ ✓

Step 3: Overflow check for the addition

- Carry into sign bit (bit 7) = 1
- Carry out of sign bit (bit 7) = 1

Rule: overflow occurs only when carry-in to the sign bit $\neq$ carry-out of the sign bit. Here both are 1, so there is NO overflow. This makes sense — one operand is positive and one is negative, and the sum of a positive and a negative number can never overflow an n-bit signed representation.

Step 4: Subtraction — (+45) - (-23)

Subtraction is implemented as addition of the 2's complement: $(+45) - (-23) = (+45) + (+23)$.

```
00101101  (+45)
+ 00010111  (+23)
-----
01000100  (68 decimal)
```

Check: $45 + 23 = 68$ ✓

Step 5: Overflow check for the subtraction

- Carry into sign bit = 0, Carry out of sign bit = 0 → equal → NO overflow

Also note both operands are positive and the result (sign bit = 0) is positive and within the representable range $[-128, +127]$, confirming no overflow.

Interpretation

Both operations produced correct, valid 8-bit signed results with no overflow, because overflow in 2's complement addition/subtraction can only occur when adding two numbers of the SAME sign yields a result of the opposite sign — which did not happen in either case here.

Question 2: 4-bit Carry Look-Ahead Adder (CLA)

Example inputs: A = 1100 (12), B = 0110 (6), C_{in} = 0. (Bits numbered A₃A₂A₁A₀ / B₃B₂B₁B₀, bit 0 = LSB.)

Step 1: Generate (G_i) and Propagate (P_i) signals

$G_i = A_i \cdot B_i$ (a carry is generated at bit i) $P_i = A_i \oplus B_i$ (a carry is propagated through bit i)

Bit i	A _i	B _i	G _i = A _i ·B _i	P _i = A _i ⊕B _i
0	0	0	0	0
1	0	1	0	1
2	1	1	1	0
3	1	0	0	1

Step 2: Compute carries in advance using the CLA equations

- $C_1 = G_0 + P_0 \cdot C_0 = 0 + 0 \cdot 0 = 0$
- $C_2 = G_1 + P_1 \cdot C_1 = 0 + 1 \cdot 0 = 0$
- $C_3 = G_2 + P_2 \cdot C_2 = 1 + 0 \cdot 0 = 1$
- C_4 (carry-out) = $G_3 + P_3 \cdot C_3 = 0 + 1 \cdot 1 = 1$

Every carry (C₁–C₄) is computed directly and simultaneously from G, P and C₀ — none of them waits for the previous stage's sum to ripple through, which is the key idea of look-ahead.

Step 3: Compute the sum bits

$S_i = A_i \oplus B_i \oplus C_i$

- $S_0 = 0 \oplus 0 \oplus 0 = 0$
- $S_1 = 0 \oplus 1 \oplus 0 = 1$
- $S_2 = 1 \oplus 1 \oplus 0 = 0$
- $S_3 = 1 \oplus 0 \oplus 1 = 0$

Result = C₄ S₃S₂S₁S₀ = 1 0010 = 18 (decimal). Check: 12 + 6 = 18 ✓

Step 4: Delay comparison — CLA vs Ripple Carry Adder (RCA)

Parameter	Ripple Carry Adder	Carry Look-Ahead Adder
Carry dependency	Each stage waits for carry from previous stage	All carries computed in parallel from G, P, C ₀
Delay (4-bit)	≈ 2n gate delays (grows with n)	≈ constant (2–3 gate delays) + 2 for sum
Hardware cost	Low	Higher (extra AND/OR gates)
Scalability	Delay grows linearly with word length	Delay grows much more slowly (needs multi-level look-ahead for wide words)

Interpretation

Because every C_i is derived directly from G, P and C₀ instead of the previous full adder's output, the worst-case path length no longer scales with the number of bits n the way it does for RCA. This is exactly why modern high-performance processors use CLA (often in a hierarchical form for wide ALUs) — it removes the ripple bottleneck at the cost of extra gates.

Question 3: Booth's Algorithm ($-6 \times +5$)

Using 4-bit 2's complement registers: Multiplicand $M = -6 = 1010$, $-M = 0110$. Multiplier $Q = +5 = 0101$, $Q_{-1} = 0$, Accumulator $A = 0000$. Number of cycles = 4 (word length).

Booth's rule reminder

- $Q_0 Q_{-1} = 10 \rightarrow A = A - M$
- $Q_0 Q_{-1} = 01 \rightarrow A = A + M$
- $Q_0 Q_{-1} = 00$ or $11 \rightarrow$ no arithmetic operation
- After every operation: arithmetic shift right the combined register A-Q-Q-1 (sign bit of A is preserved)

Step-by-step execution

Cycle	$Q_0 Q_{-1}$	Operation	A (after op)	A-Q-Q-1 after shift
1	1 0	$A = A - M$	0110	$A=0011$ $Q=0010$ $Q_{-1}=1$
2	0 1	$A = A + M$	1101	$A=1110$ $Q=1001$ $Q_{-1}=0$
3	1 0	$A = A - M$	0100	$A=0010$ $Q=0100$ $Q_{-1}=1$
4	0 1	$A = A + M$	1100	$A=1110$ $Q=0010$ $Q_{-1}=0$

The accumulator holds the running partial sum/difference, the multiplier register Q is shifted right one bit each cycle to bring in a new pair of bits, and Q_{-1} remembers the bit just shifted out — together $Q_0 Q_{-1}$ tell Booth's algorithm whether a run of 1s is starting, continuing, or ending.

Final result

After 4 cycles: $A = 1110$, $Q = 0010 \rightarrow$ Product = A Q = 1110 0010 (8-bit 2's complement).

Converting: $11100010 \rightarrow$ invert (00011101) + 1 = 00011110 = 30 $\rightarrow$ so the value is -30.

Check: $(-6) \times (+5) = -30$ ✓

Interpretation

Booth's algorithm needed only two arithmetic operations (one subtraction, one addition) instead of examining every '1' bit individually as in simple shift-add multiplication. By encoding a run of consecutive 1s as a single subtract-then-add pair, it reduces the number of add/subtract operations, and because it works directly on 2's complement operands it handles signed multiplication (like $-6 \times +5$) correctly without any special sign-correction step.

Question 4: Division of $13 \div 3$ — Restoring vs Non-Restoring

4-bit unsigned registers. Dividend $Q = 1101$ (13), Divisor $M = 0011$ (3), Accumulator $A = 00000$ (5-bit, to hold the sign during subtraction). $n = 4$ iterations.

A. Restoring Division

Rule each cycle: (1) shift the combined A-Q register left by 1 bit. (2) $A = A - M$. (3) If A is negative (sign bit = 1): set $Q0 = 0$ and RESTORE by $A = A + M$. If A is non-negative: set $Q0 = 1$ (keep A).

Cycle	After shift (A, Q)	A - M	Sign	Action	A, Q after cycle
1	00001, 1010	11110	-	$Q0=0$, restore $A=A+M$	00001, 1010
2	00011, 0100	00000	+	$Q0=1$, keep A	00000, 0101
3	00000, 1010	11101	-	$Q0=0$, restore $A=A+M$	00000, 1010
4	00001, 0100	11110	-	$Q0=0$, restore $A=A+M$	00001, 0100

Final: Quotient $Q = 0100 = 4$, Remainder $A = 00001 = 1$. Check: $3 \times 4 + 1 = 13$ ✓

B. Non-Restoring Division

Rule each cycle: if A is negative, shift A-Q left then $A = A + M$; if A is non-negative, shift left then $A = A - M$. Set $Q0 = 1$ if the new A is non-negative, else $Q0 = 0$. No immediate restore — only one correction at the very end if the final A is negative.

Cycle	Prev. sign of A	Shift then op	A result	Sign	$Q0$ set
1	+	shift, $A = A - M$	11110	-	0
2	-	shift, $A = A + M$	00000	+	1
3	+	shift, $A = A - M$	11101	-	0
4	-	shift, $A = A + M$	11110	-	0

Final A is negative → one correction: $A = A + M = 11110 + 00011 = 00001$.

Final: Quotient $Q = 0100 = 4$, Remainder $A = 00001 = 1$. Check: $3 \times 4 + 1 = 13$ ✓ (same result as restoring division)

Step: Comparison

Aspect	Restoring Division	Non-Restoring Division
Operations per cycle	Subtract, and possibly an extra restoring addition	Exactly one add OR subtract
Worst-case ops (4 bits)	Up to 8 (4 subtracts + up to 4 restores)	4 (plus at most 1 final correction) = 5
Hardware complexity	Simpler control, but wastes cycles restoring	Needs sign-tracking logic, but no wasted restore cycles
Speed	Slower (restore step adds delay)	Faster — preferred in modern processors

Interpretation

Both methods give the same correct quotient (4) and remainder (1). Non-restoring division is generally preferred in hardware because it never 'wastes' a cycle restoring the accumulator — it simply remembers the sign and picks add or subtract on the

next cycle, needing at most one corrective addition at the end. This makes it faster and more efficient than restoring division for the same result.

Question 5: IEEE 754 Representation & Addition (-13.25)

Step 1: Convert -13.25 to binary

- 13 (integer part) = 1101
- 0.25 (fraction part) = 0.01
- So $13.25 = 1101.01_2$, and the sign is negative

Step 2: Normalize

$$1101.01 = 1.10101 \times 2^3 \text{ (moved the binary point 3 places left, so exponent = 3)}$$

Step 3: Single precision (32-bit) fields

- Sign (1 bit) = 1 (negative)
- Exponent (8 bits) = actual exponent + bias(127) = 3 + 127 = 130 = 10000010
- Mantissa/Fraction (23 bits) = the bits after the leading 1: 10101 followed by zero-padding = 10101000000000000000000

Single precision bit pattern: 1 1000010 1010100000000000000000

Step 4: Double precision (64-bit) fields

- Sign (1 bit) = 1
- Exponent (11 bits) = $3 + \text{bias}(1023) = 1026 = 10000000010$
- Mantissa/Fraction (52 bits) = 10101 followed by zero-padding to 52 bits

Double precision bit pattern: 1 10000000010
10101000

Step 5: Floating-point addition example ($-13.25 + 5.75$)

To illustrate FP addition, add -13.25 to 5.75 ($5.75 = 101.11_2 = 1.0111 \times 2^2$).

Step 1: Align exponents to the larger one (3): $1.0111 \times 2^2 = 0.10111 \times 2^3$.

Step 2: Since the signs differ, subtract magnitudes: $1.10101 - 0.10111 = 0.11110$ (result stays negative, matching the larger-magnitude operand).

Step 3: Result so far: -0.11110×2^3 — the leading bit is 0, so it is NOT normalized.

Step 4: Normalize: shift the mantissa left by 1 and decrease the exponent by 1 $\rightarrow -1.1110 \times 2^2$.

Step 5: Convert back to decimal: $-1.1110 \times 2^2 = -111.10_2 = -7.5$

Check: $-13.25 + 5.75 = -7.5 \checkmark$

Interpretation

This example required a post-subtraction normalization step (shifting the mantissa and adjusting the exponent) because subtracting two aligned mantissas can produce leading zeros. In general, IEEE-754 addition/subtraction can also require rounding when the result's mantissa needs more bits than the format allows (23 for single, 52 for double); the extra bits are rounded off using guard, round, and sticky bits, which is the source of precision loss / rounding error in floating-point arithmetic. Double precision keeps far more mantissa and exponent bits than single precision, so it represents the same value with much higher precision and a wider exponent range — important for scientific computing where accumulated rounding error must stay small.